

主流程完成后的 OPTIONAL PATH

SmolVLA 提升选做手册

从 SmolVLA Base 训练 LIBERO 数据集并评估新检查点

定位：科研训练、消融对比与 LIBERO 能力提升；不是基础环境部署必做项。

编写原则：原始命令与执行顺序完全保留，仅补充选择依据、收益、成本和验收标准。

先判断

这条路线解决的是“从 SmolVLA Base 出发，在 LeRobot 格式的 LIBERO 示范数据上重新训练，并验证训练后模型的仿真成功率”。它补上了主手册中没有执行的 LIBERO 数据集训练环节，但不会替代主手册的环境准备、官方策略基线评估和 Piper 真机微调。

建议选做

- 主手册的 `smolvla_libero` 基线评估已经跑通，能够确认 EGL、assets、相机键和控制模式没有问题。
- 研究目标包含 LIBERO 复现、基础模型微调、训练前后对比或不同冻结策略的消融实验。
- 云服务器具备足够的 GPU 时间、显存、数据盘空间，并能承担 25,000 步训练和 400 回合评估。
- 需要一个由自己训练产生、输出目录和检查点都可追溯的 LIBERO 模型。

推荐顺序：先完成主手册并保存 `smolvla_libero` 的基线结果，再执行本手册；训练结束后使用完全相同的任务套件、relative 控制模式和每任务 10 回合设置比较成功率。

成本提示：本手册会更新视觉编码器且不限为只训练动作专家，属于计算和显存开销更高的完整微调路线。它适合作为提升实验，不应被误解为主流程的前置条件。

第一部分 选做路线的运行准备

复用主手册已经跑通的 LeRobot、SmolVLA、SmolVLM2、LIBERO assets 与 EGL 环境，只新增 LIBERO 训练数据集和更偏向完整微调的运行参数。

步骤 1 设置训练与评估全局变量

执行位置：AutoDL 云服务器；与主手册相同的 lerobot Conda 环境

选做目标：统一 Hugging Face 缓存、LIBERO 数据集、LIBERO assets、SmolVLA Base、SmolVLM2 和 GPU 运行参数。

LIBERO_DATASET 指向本地 LeRobot 数据集目录；路径中的 lerobot/lerobot/libero 分别对应缓存根目录、仓库命名空间和数据集名称，并不一定是误写的重复目录。CUDA_VISIBLE_DEVICES=0 固定使用第一块可见 GPU；expandable_segments 用于缓解显存碎片；OMP_NUM_THREADS=1 限制 CPU 线程竞争。所有 export 只在当前终端会话生效。

原始命令（保持不变）

#设置全局变量

```
conda activate lerobot
```

```
export HF_HOME=/root/autodl-tmp/cache/huggingface
```

```
export HF_LEROBOT_HOME=/root/autodl-tmp/huggingface/lerobot
```

```
export LIBERO_DATASET=/root/autodl-tmp/huggingface/lerobot/lerobot/libero
```

```
export LIBERO_ASSETS=/root/autodl-tmp/libero/assets
```

```
export SMOLVLA_BASE=/root/autodl-tmp/pretrained/smolvla_base
```

```
export SMOLVLM_DIR=/root/autodl-tmp/cache/huggingface/manual_models/SmolVLM2-500M-Video-Instruct
```

```
export CUDA_VISIBLE_DEVICES=0
```

```
export TOKENIZERS_PARALLELISM=false
```

```
export PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True
```

```
export OMP_NUM_THREADS=1
```

完成标志：Conda 环境成功激活，变量展开为预期绝对路径，GPU 0 可见；SMOLVLA_BASE、SMOLVLM_DIR 与 LIBERO_ASSETS 已由主手册准备完整。

注意：本选做手册不重复安装 LeRobot、EGL 或模型文件。若主手册的 LIBERO 基线评估尚未成功，应先回到主手册排除环境问题，否则训练问题和环境问题会混在一起。

第二部分 下载 LIBERO 数据并训练 SmolVLA Base

从镜像获取 LeRobot 格式的 LIBERO 数据集，然后回到离线模式，对基础模型进行 25,000 步完整微调。

步骤 2 通过镜像下载 LeRobot 格式的 LIBERO 数据集

执行位置：AutoDL 云服务器（临时联网阶段）

选做目标：退出三个离线模式，配置 Hugging Face 镜像，并把 lerobot/libero 数据集下载到 LIBERO_DATASET。

--repo-type dataset 明确下载对象是数据集仓库；--local-dir 把数据放到训练命令将要读取的本地根目录。已有的完整文件通常会被校验并复用，未完成文件可继续下载。HF_HUB_DISABLE_XET=1 避开 Xet 下载链路，600 秒超时适合大体积数据。

原始命令（保持不变）

#通过镜像下载 libero 数据集

unset HF_HUB_OFFLINE

unset TRANSFORMERS_OFFLINE

unset HF_DATASETS_OFFLINE

export HF_ENDPOINT=https://hf-mirror.com

export HF_HUB_DISABLE_XET=1

export HF_HUB_DOWNLOAD_TIMEOUT=600

mkdir -p "\$LIBERO_DATASET"

hf download lerobot/libero \

--repo-type dataset \

--local-dir "\$LIBERO_DATASET"

完成标志：LIBERO_DATASET 中存在 meta/info.json、数据文件和相应视频资源，下载过程没有残留失败或不完整的大文件。

说明：LIBERO 训练数据与 LIBERO assets 不是同一类资源：前者提供离线示范轨迹用于训练，后者提供 MuJoCo 场景、物体和纹理用于仿真。二者都需要，但不能互相替代。

步骤 3 切回离线模式并创建独立训练目录

执行位置：AutoDL 云服务器；继续使用同一终端

选做目标：关闭 Hub、Transformers 和 Datasets 的联网访问，取消镜像端点，并用时间戳生成唯一训练输出目录。

RUN_TAG 使用当前日期和时间，避免不同训练轮次写入同一目录。TRAIN_OUT 将成为训练日志、配置与检查点的共同根目录；echo 负责在长时间训练开始前把真实路径打印出来，便于记录实验。

原始命令（保持不变）

```
#训练
export HF_HUB_OFFLINE=1
export TRANSFORMERS_OFFLINE=1
export HF_DATASETS_OFFLINE=1

unset HF_ENDPOINT
RUN_TAG=$(date +%Y%m%d_%H%M%S)
TRAIN_OUT=/root/autodl-tmp/outputs/smolvla_base_libero_${RUN_TAG}

echo "TRAIN_OUT=$TRAIN_OUT"
```

完成标志：终端打印一个带时间戳的 TRAIN_OUT 路径，且后续模型加载不再尝试联网。

步骤 4 在 LIBERO 数据集上完整微调 SmolVLA Base

执行位置：AutoDL 云服务器 GPU；高计算开销选做项

选做目标：使用本地 LIBERO 数据和 smolvla_base 初始化权重训练 25,000 步，同时更新视觉编码器和动作相关模块。

dataset.root 指向本地数据，dataset.repo_id 保留数据集身份；pyav 用于读取视频。

use_imagenet_stats=false 不强制采用 ImageNet 图像统计量，实际预处理仍由策略与数据集配置决定。

load_vlm_weights=true 加载 SmolVLM2 权重；freeze_vision_encoder=false 允许视觉编码器更新；

train_expert_only=false 表示不局限于只训练动作专家，适应能力更强，但显存、训练时间和过拟合风险

也更高。empty_cameras=1 增加被掩蔽的占位视觉槽位。学习率调度先预热 1,000 步，再在 25,000 步内

衰减到 2.5e-6。rename_map 按“数据集源键 → 策略目标键”把 image/image2 映射为

camera1/camera2。训练每 5,000 步保存一次，不在训练期间启动仿真评估，也不上传 W&B 或 Hub。

原始命令（保持不变）

```
lerobot-train \
  --dataset.repo_id=lerobot/libero \
  --dataset.root="$LIBERO_DATASET" \
  --dataset.video_backend=pyav \
  --dataset.use_imagenet_stats=false \
  --policy.path="$SMOLVLA_BASE" \
```

```

--policy.device=cuda \
--policy.load_vlm_weights=true \
--policy.freeze_vision_encoder=false \
--policy.train_expert_only=false \
--policy.empty_cameras=1 \
--policy.scheduler_warmup_steps=1000 \
--policy.scheduler_decay_steps=25000 \
--policy.scheduler_decay_lr=2.5e-6 \
--
rename_map='{ "observation.images.image": "observation.images.camera1", "observation.images.image2": "observation.images.camera2" }' \
--output_dir="$TRAIN_OUT" \
--job_name=smolvla_base_libero \
--batch_size=4 \
--num_workers=4 \
--steps=25000 \
--save_freq=5000 \
--log_freq=100 \
--env_eval_freq=0 \
--wandb.enable=false \
--policy.push_to_hub=false

```

完成标志：训练日志持续输出 loss，TRAIN_OUT 下出现 005000、010000、015000、020000、025000 或 last 检查点，最终 pretrained_model 内含 model.safetensors。

注意：这是面向 LIBERO 仿真能力的完整微调，并不直接提升 Piper 真机任务。若研究目标只是香蕉抓取或 Piper 抗干扰，应优先把时间投入真机数据质量、任务变化覆盖和推理链路，而不是默认把 LIBERO 训练当作必做项。

第三部分 选择检查点并进行标准评估

自动选择 last 或 025000 检查点，配置云服务器 EGL 环境，并执行四套件、每任务 10 回合的标准评估。

步骤 5 自动选择可用于评估的最终检查点

执行位置：AutoDL 云服务器；训练结束后

选做目标：优先使用 checkpoints/last/pretrained_model；如果 last 中不存在 model.safetensors，则回退到 025000/pretrained_model。

文件存在性判断只检查 model.safetensors。echo 显示最终采用的检查点路径，ls -lh 同时验证权重存在并显示体积。如果两个候选目录都不完整，ls 会直接报错，评估不应继续。

原始命令（保持不变）

```
#仿真
SMOLVLA_CKPT="$TRAIN_OUT/checkpoints/last/pretrained_model"

if [ ! -f "$SMOLVLA_CKPT/model.safetensors" ]; then
  SMOLVLA_CKPT="$TRAIN_OUT/checkpoints/025000/pretrained_model"
fi

echo "$SMOLVLA_CKPT"
ls -lh "$SMOLVLA_CKPT/model.safetensors"
```

完成标志：echo 输出正确的 SMOLVLA_CKPT，ls 能列出非空的 model.safetensors。

说明：last 通常代表训练程序维护的最终检查点入口；025000 是本合同 25,000 步训练的显式编号检查点。回退逻辑能处理不同保存结构，但不能修复训练中断或权重损坏。

步骤 6 配置评估 GPU、EGL 与离线模式

执行位置：AutoDL 云服务器

选做目标：指定 MuJoCo/PyOpenGL 使用第 0 块 GPU 的 EGL 无界面渲染，并再次确保模型与数据完全离线加载。

CUDA_VISIBLE_DEVICES=0、MUJOCO_EGL_DEVICE_ID=0 和 EGL 平台变量应与实际 GPU 可见顺序一致。TOKENIZERS_PARALLELISM=false 减少多进程提示；三个 OFFLINE=1 变量用于验证所有资源都已本地化。

原始命令（保持不变）

```
export CUDA_VISIBLE_DEVICES=0
export MUJOCO_GL=egl
export PYOPENGL_PLATFORM=egl
export MUJOCO_EGL_DEVICE_ID=0
export TOKENIZERS_PARALLELISM=false

export HF_HUB_OFFLINE=1
export TRANSFORMERS_OFFLINE=1
export HF_DATASETS_OFFLINE=1
```

完成标志：创建 LIBERO 环境时不出现 eglQueryString、EGL cannot run、缺少 assets 或联网下载错误。

步骤 7 评估新训练的 LIBERO 检查点

执行位置：AutoDL 云服务器 GPU；长时间评估选做项

选做目标：使用带时间戳的独立输出目录，对 Spatial、Object、Goal 和 LIBERO-Long 四套件执行标准成功率评估。

policy.path 指向本轮训练检查点，而不是下载好的 smolvla_libero。relative 控制模式必须与策略动作表示一致；batch_size=1、max_parallel_tasks=1、use_async_envs=false 采用稳健的串行评估。四个套件各 10 个任务，每个任务 10 回合，总计 400 回合。rename_map 与训练保持一致，保证环境的 image/image2 被策略按 camera1/camera2 接收。时间戳使每次评估结果独立保存，便于与主手册中的基线策略做同条件比较。

原始命令（保持不变）

#评估

```
RUN_TAG=$(date +%Y%m%d_%H%M%S)
```

```
EVAL_ALL=/root/autodl-tmp/eval_logs/smolvla_trained_all_${RUN_TAG}
```

```
lerobot-eval \
```

```
--output_dir="$EVAL_ALL" \
```

```
--policy.path="$SMOLVLA_CKPT" \
```

```
--policy.device=cuda \
```

```
--policy.empty_cameras=1 \
```

```
--env.type=libero \
```

```
--env.task=libero_spatial,libero_object,libero_goal,libero_10 \
```

```
--env.control_mode=relative \
```

```
--env.max_parallel_tasks=1 \
```

```
--eval.batch_size=1 \
```

```
--eval.n_episodes=10 \
```

```
--eval.use_async_envs=false \
```

```
--
```

```
rename_map='{ "observation.images.image": "observation.images.camera1", "observation.images.image2": "observation.images.camera2" }'
```

完成标志：EVAL_ALL 下生成日志和回放文件，结果包含各套件成功率、整体 pc_success 与耗时；可在同样的四套件、relative 模式和 10 回合设置下，与 smolvla_libero 基线对比。

注意：400 回合评估耗时较长。只有在模型能正常加载、单任务短测不报错、输出目录和检查点确认无误后，才值得运行完整评估。成功率提高也只说明 LIBERO 条件下提升，不能直接外推到 Piper 真机。

如何判定这次选做是否带来提升

- 可运行性：训练检查点能离线加载，LIBERO 环境能连续完成评估，无缺失键、权重或 EGL 错误。

- 同条件比较：基线与新模型必须使用相同四套件、relative 控制模式、每任务 10 回合及相同相机映射。
- 分套件观察：分别记录 Spatial、Object、Goal、LIBERO-Long，不能只看整体平均值。
- 稳定性：如果只在某一次评估中提高，应结合多次训练或重复评估判断是否为随机波动。
- 边界：LIBERO 成功率只代表仿真基准表现，不能作为 Piper 真机成功率的替代指标。

本路线相对主手册新增了什么

- 新增 lerobot/libero 离线示范数据集下载，而不只是下载 LIBERO 场景 assets。
- 从 smolvla_base 出发训练 25,000 步，而不直接使用下载好的 smolvla_libero。
- 允许视觉编码器更新，并且不限定为 train_expert_only，属于更完整的参数适配。
- 使用时间戳保存每次训练和评估，便于形成可追溯的实验记录。
- 评估对象改为本轮训练产生的 SMOLVLA_CKPT，可与官方策略基线进行同条件对照。

参考依据

说明文字依据 LeRobot v0.6.0 官方文档及原始命令上下文整理；命令本体来源于用户上传的 smovla1.docx，未作修改。

SmolVLA: <https://huggingface.co/docs/lerobot/v0.6.0/smolvla>

LIBERO: <https://huggingface.co/docs/lerobot/v0.6.0/libero>

Rename Map and Empty Cameras: https://huggingface.co/docs/lerobot/v0.6.0/rename_map